

ESILV Smart Assistant

Document-grounded QA System
(RAG)

Built for Students & Applicants

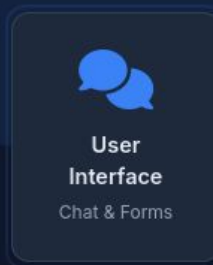
Providing accurate answers on programs, admissions, rules, and calendars grounded in official documents.

 FastAPI

 React

 FAISS

 Ollama



Agenda

Overview of ESILV Smart Assistant System



01

Project Context



02

Objectives & Goals



03

High-Level Architecture



04

Ingestion Pipeline



05

RAG Online Flow



06

Multi-Agent Design



07

Backend API & UI



08

Results & Value



09

Demo Plan



10

Limitations & Challenges



11

Future Work



Objectives

Goals & Success Criteria for the Smart Assistant



Efficient Discovery

Reduce time to find information buried across disconnected PDFs, HTML pages, and text files.



Grounded Accuracy

Increase correctness through RAG retrieval and explicit source citations, minimizing hallucinations.



Intelligent Routing

Support multiple user intents (Info vs. Admin) via a multi-agent orchestrator system.



Usable Interfaces

Provide a robust FastAPI backend with a React frontend, plus optional demo tools.



Reliable Engineering

Maintain high test coverage with mocked LLM calls, avoiding live dependency for CI/CD.



SPEED

Fast retrieval from indexed documents



TRUST

Citations & source transparency



EXPERIENCE

Seamless chat & form interface



SCALE

Modular agent architecture

High-Level System Architecture

Modular Design: Ingestion, Agents, & Serving Layers



Serving Layer (UI/API)

FastAPI backend exposing REST endpoints combined with a React frontend for chat, uploads, and admin management.



Agent Layer (Logic)

Central Orchestrator routes requests to specialized agents: Retrieval (RAG), Form (Lead Gen), and FAQ.



Ingestion Layer (Offline)

Processes raw PDF/HTML documents: loads, cleans, chunks, embeds (MiniLM), and builds the FAISS index.



Data Artifacts

Canonical outputs: `index.faiss` for vector search and `index_metadata.json` for source tracking.



SERVING / UI LAYER



React Frontend



FastAPI Backend



AGENT LAYER



Orchestrator

Retrieval

Form

FAQ



INGESTION LAYER (OFFLINE)

Ingestion Pipeline

Offline / Periodic Document Processing

BATCH PROCESS



CLI EXECUTION

```
# Run ingestion pipeline
python -m ingestion.pipeline \
  --data-dir data/raw \
  --output data/vector_db/index
```

GENERATED ARTIFACTS

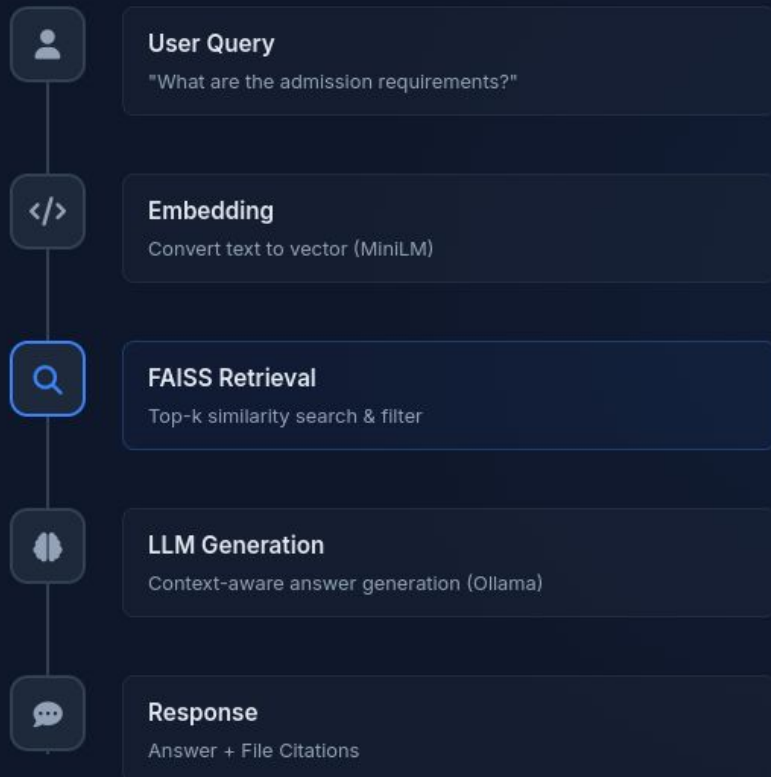
 index.faiss

 index_metadata.json

Retrieval-Augmented Generation

Online Query Execution Flow

● ONLINE SYSTEM



GROUNDING SAFEGUARDS

- Similarity Threshold:** Discard chunks with score < 0.5 to reduce noise.
- Fallback Behavior:** If no context found, explicitly reply "Unknown" rather than hallucinating.

>_ SYSTEM PROMPT

```
# Instruction
You are a helpful assistant.
Use ONLY the provided context to answer.
If the answer is not in context, say "I don't know".
```

” Output Format Example

"Students must validate B2 level in English..."

Source 1 rules_2024.pdf (Page 12)

Multi-Agent Design

Intelligent Routing & Specialized Processing Architecture

 Distributed Logics



Orchestrator Agent

Classifies user intent from natural language query.

LLM Classification

Routes request to appropriate handler based on intent probability.



Retrieval Agent

Handles informational queries using vector search + grounded generation.

RAG Pipeline

Citations



Form Agent

Captures structured data (name, email, interest) and persists leads.

JSON Storage

Slot Filling



FAQ Agent

Provides deterministic answers for common high-frequency questions.

Token Overlap

Fast Path

Backend API + UI Integration

RESTful Endpoints & React Frontend Architecture



Chat Interaction

POST /api/chat

Main orchestrator entry point. Receives query, routes to agents, returns structured answer with sources.



Document Ingestion

POST /api/upload

Saves files to raw storage and triggers ingestion pipeline logic.



Admin & Maintenance

POST /api/admin/reindex

GET /api/admin

Rebuild FAISS index, list current uploads, and view captured leads.



Frontend Config

React SPA configured via VITE_API_URL. Features modular Tabs for distinct user flows (Chat vs Admin).



Results & Value Delivered

Impact of the RAG Implementation

✓ SYSTEM OPERATIONAL



End-to-End Pipeline



Fully automated workflow from raw document upload to vector indexing, enabling seamless knowledge updates.

Ingestion Layer

Offline Processing



Grounded Accuracy



Responses are strictly grounded in ESILV documents, significantly reducing hallucinations compared to generic LLMs.

Context Injection

Prompt Engineering



Source Traceability



Every answer includes file-level citations, allowing students and applicants to verify information sources.

Metadata Tracking

Trust & Safety



Robust Engineering



High testing coverage for ingestion components and agent routing logic, with LLM calls mocked for reliability.

Unit Tests

CI/CD Ready

Mocking

⌚ Reduced Information Retrieval Time

✓ Deterministic Fallback Mechanisms

📁 Clear Separation of Concerns

Demo Plan (Live Steps)

End-to-End System Walkthrough

 LIVE DEMO

1

Start Environment

Launch backend API and frontend servers locally.

```
uvicorn backend.main:app --reload --port 8001
```

```
npm run dev
```

2

Document Upload

Navigate to Upload tab and ingest a sample PDF document into the raw data directory.

3

Trigger Re-indexing

Initiate the FAISS index build process via Admin tab or API endpoint.

```
POST /api/admin/reindex
```

```
Generate Embeddings
```

4

Interactive QA

Ask a specific question in Chat tab to demonstrate retrieval-augmented generation.

```
✓ Verify Grounding
```

```
🗉 Check Citations
```



Limitations & Challenges

Current Constraints and Technical Hurdles

INTELLIGENCE & EXTRACTION

DATA QUALITY



PDF Extraction Variability

Parsing complex document structures remains challenging. Scanned PDFs, multi-column layouts, and embedded tables often result in noisy text chunks.

⚠ Issue: Header/Footer leakage in chunks

LATENCY



LLM Dependency & Latency

Routing logic relies on LLM classification, creating a hard dependency on Ollama availability. Inference time impacts the

ARCHITECTURE & OPS

CONSISTENCY



Index Synchronization

The core retrieval index and the admin stub index (for listing uploads) must be kept in sync manually. Desynchronization leads to "ghost" documents.

⚠ Risk: Retrieval index vs Admin DB drift

OPERATIONS



Operational Overhead

No automated monitoring for reindex jobs. Artifact versioning (FAISS indices) is currently handled via file overwrites rather than

Future Work & Improvements

Roadmap for functionality, reliability, and performance



Heuristic-First Routing

Detect simple intents (e.g., forms, emails) using regex/keywords before invoking the LLM to reduce latency and cost.



Granular Citations

Surface precise page numbers and chunk IDs in the UI to improve trust and traceability of answers.



Incremental Indexing

Process individual file uploads and updates without rebuilding the entire FAISS index from scratch.



OCR & Advanced Extraction

Integrate OCR for scanned PDFs and improve logic for extracting structured data from complex tables.



Automated Benchmarking

Implement a harness to measure Recall@k, MRR, and faithfulness against a golden dataset.

NOW



OPTIMIZATION (NEAR-TERM)

 Heuristic Routing

 Chunk-level Citations

 Error Handling

NEXT



ENGINEERING (MID-TERM)

 Incremental Indexing

 Async Processing

 Metadata Filtering

LATER



CAPABILITIES (LONG-TERM)

 OCR for Scans

 Table Parsing

 RAG Benchmark Harness